School: Mittal School of Business	**Name of Faculty Member:**
Course Code: int234	**Task no.: 2**
Course Title:	
Date of Allotment:	**Roll No.: 54**
Date of Submission: 29 october	**Registration No.: 12410698**
Term:	**Maximum Marks: 30**

Title:

Declaration: I declare that this assignment is my individual work and I have not copied it from any other student's work or from any other source, nor has any part been written by another person.

Signature: sparsh dadwal

CERTIFICATE

This is to certify that Sparsh Dadwal bearing Registration no. 12410698 has completed project titled, **"CPI Data Analysis and Prediction"** under my guidance and supervision. To the best of my knowledge, the present work is the result of his/her original development, effort and study.

Signature and Name of the Supervisor
Designation of the Supervisor
School of BTECG CSE
Lovely Professional University
Phagwara, Punjab.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project supervisor, for the guidance, encouragement and valuable suggestions that helped me to complete this project. I am also thankful to the faculty members of at Lovely Professional University for providing the academic foundation in Data Science and Python programming on which this work is based.

I acknowledge the Ministry of Statistics and Programme Implementation, Government of India, for making the Consumer Price Index data available to the public, and the developers of the open-source Python libraries Pandas, NumPy, Matplotlib, Seaborn and Scikit-learn, without which this project could not have been carried out. Finally, I thank my family and friends for their continuous support.

Date: 29 october 2026 SPARSH DADWAL

TABLE OF CONTENTS

1. Introduction 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope and Limitations of the Study 2
1.5 Tools and Technologies Used 2
1.6 Organisation of the Report 3
2. Source of Dataset 4
2.1 Origin of the Data 4
2.2 Structure of the File 4
2.3 Size and Coverage of the Data 5
2.4 The Two Variables of Interest 5
2.5 Interpreting Index Values 5
3. EDA Process 7
3.1 Overview of the Workflow 7
3.2 Loading and Inspecting the Data 7
3.3 Dropping the Inflation Columns 9
3.4 Filtering Rows with index_al > 200 9
3.5 Checking Missing Values 9
3.6 Handling Zero Values 10
3.7 Checking Duplicate Rows 10
3.8 Train–Test Split and Feature Scaling 10
3.9 Summary of Pre-processing 11
4. Analysis on Dataset 12
4.1 Relationship between Rural CPI and Agricultural CPI 12
4.2 Top Five States by Maximum Agricultural CPI 13
4.3 Distribution of Agricultural CPI 15
4.4 Year-wise Trend of Average Agricultural CPI 16
4.5 Three-Year Moving Average of Agricultural CPI 18
4.6 Minimum versus Maximum Agricultural CPI by Year 19
4.7 Top versus Bottom State by Mean Agricultural CPI 21
4.8 Regression Set-up and Evaluation Metrics 23
4.9 Linear Regression 25
4.10 Decision Tree Regression 26
4.11 K-Nearest Neighbors (KNN) Regression 28

4.12 Support Vector Regression (SVR) 29
4.13 Comparison of the Four Models 31
5. Conclusion 35
5.1 Summary of Findings 35
5.2 Limitations and Methodological Observations 35
6. Future Scope 37
Appendix A: Python Source Code 38
7. References 44

LIST OF FIGURES

Figure 1: Dataset shape, column names and missing-value counts printed by the project code8
Figure 2: Last five rows of the dataset (data.tail())....8
Figure 3: Scatter plot of Rural CPI versus Agricultural CPI (regenerated from the project code and the supplied dataset) 13
Figure 4: Top five states with the highest maximum Agricultural CPI (output of the project code)....15
Figure 5: Histogram of Agricultural CPI (regenerated from the project code and the supplied dataset) 16
Figure 6: Year-wise trend of average Agricultural CPI (regenerated from the project code and the supplied dataset)....17
Figure 7: Three-year moving average of Agricultural CPI (regenerated from the project code and the supplied dataset)....19
Figure 8: Minimum versus maximum Agricultural CPI by year (output of the project code) 21
Figure 9: Top versus bottom state by mean Agricultural CPI (output of the project code) 23
Figure 10: Console output for Linear Regression, Decision Tree and KNN Regression 26
Figure 11: Decision Tree Regression plot produced by plot_tree() (output of the project code) 28
Figure 12: SVR: actual versus predicted Agricultural CPI for the test set (output of the project code)....31
Figure 13: Console output for Support Vector Regression and the final model comparison ..31
Figure 14: MAE and MSE of the four models (plotted from the results in Table 12)....34

LIST OF TABLES

Table 1: Python libraries used in the project 3
Table 2: Columns of the CPI dataset 4
Table 3: Summary of the dataset size and coverage 5
Table 4: Descriptive statistics of the retained data 9
Table 5: Summary of the pre-processing steps implemented in the project 11
Table 6: Five highest state-wise maximum Agricultural CPI values 14
Table 7: Yearly average, minimum, maximum and range of Agricultural CPI 20
Table 8: Mean Agricultural CPI of all 21 entries, in descending order 22
Table 9: Configuration of the four regression models 24
Table 10: Predictions for the normalised Rural CPI input 0.5 (Rural CPI = 900.5) 32
Table 11: First four test predictions compared with the actual Agricultural CPI 32
Table 12: Final comparison of the four regression models on the test set 33
Table 13: Quantities derived from the reported metrics 33

1. Introduction

1.1 Background

The Consumer Price Index (CPI) is one of the most widely used statistical indicators of an economy. It tracks the change, over time, in the prices of a fixed basket of goods and services consumed by a defined group of households. Because the index is expressed relative to a base period, it allows the movement of prices to be compared across years, months and regions without having to compare raw rupee prices of individual items. Governments, central banks, researchers and businesses use CPI figures to monitor price movements, to adjust wages and allowances, and to understand how the cost of living changes for different sections of society.

In India, consumer price indices are published for several population groups and for several geographical levels. The dataset used in this project contains two such series for each state and for the All-India level: an Agricultural CPI (stored in the column *index_al*) and a Rural CPI (stored in the column *index_rl*). Both are recorded month by month, which makes the dataset suitable for studying how price indices evolved over a long period and how different states compare with one another.

Data Science provides a set of practical techniques for studying such data. Exploratory Data Analysis (EDA) helps a student to understand the structure, quality and behaviour of a dataset before any model is built. Data visualisation turns thousands of numbers into patterns that can be read at a glance. Machine learning regression models then make it possible to express the relationship between variables in a form that can be used to estimate one quantity from another. This project combines these three activities in a single Python workflow applied to CPI data.

1.2 Problem Statement

The project addresses the following problem: given monthly state-wise CPI records, how are the Rural CPI and the Agricultural CPI related, how have the Agricultural CPI values varied across years and across states, and how well can common regression algorithms estimate the Agricultural CPI when only the Rural CPI is supplied as an input? It is important to note that the models developed in this project estimate the Agricultural CPI of a record from the Rural CPI of the same record. They are not time-series forecasting models and the project does not claim to predict the CPI of future months or years.

1.3 Objectives

The objectives of the project, as defined at the start of the work, are listed below.

- To analyse the Consumer Price Index data obtained from the Ministry of Statistics and Programme Implementation (MOSPI), Government of India.
- To clean and pre-process the dataset so that it is suitable for analysis and modelling.
- To explore the relationship between the Rural CPI and the Agricultural CPI.
- To identify the states with high and low Agricultural CPI values.
- To study the distribution of the Agricultural CPI and its year-wise trend.
- To train regression models that estimate the Agricultural CPI from the Rural CPI.
- To compare the predictive performance of four regression models using MAE, MSE and the R^2 score.

1.4 Scope and Limitations of the Study

The scope of the project is limited to the two index columns, the year, the month and the state recorded in the supplied file. The regression stage uses a single input feature (the Rural CPI) and a single target (the Agricultural CPI). The performance figures reported in this document are measured on one 80:20 random train–test split of the data. The project does not perform cross-validation or systematic hyper-parameter tuning, and it does not attempt to forecast future index values. These limitations are discussed further in Chapter 5 so that the conclusions of the report are read in the correct context.

1.5 Tools and Technologies Used

The entire project was implemented in the Python programming language. Table 1 lists the libraries used and the role each library plays in the project [2]–[6].

Table 1: Python libraries used in the project

Library	Role in the project
Pandas	Loading the CSV file, filtering rows, grouping by year and state, summary statistics and missing-value checks
NumPy	Numerical operations and the representation of missing values (np.nan)
Matplotlib	Creating histograms, line plots, bar charts, the scatter plot and the decision-tree figure
Seaborn	Scatter plot and the bar chart of the top five states
Scikit-learn	Train–test split, MinMaxScaler, the four regression models and the evaluation metrics (MAE, MSE, R^2)

1.6 Organisation of the Report

Chapter 2 describes the source and structure of the dataset. Chapter 3 explains the EDA and pre-processing steps that were actually implemented in the code. Chapter 4 presents the individual analyses: seven exploratory analyses followed by the regression modelling stage, in which each of the four models and their comparison is described. Each analysis is written in the format prescribed by the university, namely Introduction, General Description, Specific Requirements, Functions and Formulas, Analysis Results and Visualization. Chapter 5 gives the conclusion together with the limitations of the work, Chapter 6 suggests the future scope, and the complete Python code is reproduced in Appendix A. The list of references appears at the end of the report.

2. Source of Dataset

2.1 Origin of the Data

The dataset used in this project is a Consumer Price Index file named *cpialrl_5.csv*. According to the project details supplied by the author, the data were obtained from the Ministry of Statistics and Programme Implementation (MOSPI), Government of India. The exact web page from which the file was downloaded is https://www.mospi.gov.in/ and the date of download is [insert date of download]. The file is a public statistical release and contains no personal information.

The two index columns carry the suffixes *al* and *rl*, which correspond to the Agricultural CPI and the Rural CPI used throughout this report. The file itself does not describe the compilation method of these indices, so this report does not attempt to describe how they were computed by the publishing agency.

2.2 Structure of the File

The raw file contains nine columns. Seven of these are used in the project; the two inflation columns are removed during pre-processing, as explained in Chapter 3. Table 2 describes each column based on the values observed in the file.

Table 2: Columns of the CPI dataset

Column	Meaning	Values observed in the file
indicator	Type of index recorded	Only one value: "General Index"
base_year	Base period of the index series	"1986-1987" for the rows used in the analysis; "2019" for a separate group of rows that is removed
year	Year label of the record	Year ranges such as "1999-2000" to "2024-2025" for the retained rows; the removed rows carry the labels "2025" and "2026"
month	Month of the record	Month names, January to December
state	State or region	State names and the aggregate entry "All India"
index_al	Agricultural CPI (target variable)	Numeric index values
index_rl	Rural CPI (input feature)	Numeric index values
inflation_al, inflation_rl	Inflation values for the two series	Filled only for the base-2019 rows; empty for every retained row

2.3 Size and Coverage of the Data

When the workbook version of the file was inspected, the raw data contained 6,811 rows and 9 columns. After the project removes the inflation columns and filters the rows (Section 3.4), 6,531 rows and 7 columns remain; this matches the shape printed by the project code (Figure 1). The retained data cover 26 year labels, from 1999-2000 to 2024-2025, and 21 geographical entries: 20 states or regions and the All-India aggregate. Each entry has 311 monthly records. Every year contains 12 months for every entry except 2024-2025, in which the month of June is absent, so that this final year contains 11 months of observations. Table 3 summarises these facts.

Table 3: Summary of the dataset size and coverage

Item	Value
Rows and columns in the raw file	6,811 rows × 9 columns
Rows removed by the filter index_al > 200	280 (all belong to the base-year 2019 series)
Rows and columns after pre-processing	6,531 rows × 7 columns
Year labels in the retained data	26 (1999-2000 to 2024-2025)
Geographical entries	21 (20 states or regions and All India)
Records per geographical entry	311
Months in the final year (2024-2025)	11 (June absent)

2.4 The Two Variables of Interest

The **Agricultural CPI (index_al)** is the target variable of the regression stage and the main subject of the exploratory analyses. The **Rural CPI (index_rl)** is the input feature of the regression models. In the retained data the Agricultural CPI ranges from 274 to 1551 and the Rural CPI ranges from 276 to 1532. The two columns therefore occupy almost the same numerical range, which is an early sign that they are closely related; this is examined in Section 4.1.

2.5 Interpreting Index Values

A CPI value is not a price. It shows how much the price level has changed relative to the base period of that particular series. For this reason, a higher index value for one state than for another indicates a larger cumulative rise in prices since the base period in that state, and it does not by itself show that goods are more expensive in absolute terms in that state. Likewise,

the yearly rise of the index shown in later chapters reflects the cumulative nature of the index and should not be read as an annual inflation rate. This distinction is kept in mind when the figures are interpreted.

3. EDA Process

3.1 Overview of the Workflow

The exploratory data analysis and pre-processing carried out in the project follow the sequence below. Each step corresponds to a block of code reproduced in Appendix A.

1. Load the CSV file into a Pandas DataFrame.
2. Drop the columns inflation_al and inflation_rl.
3. Keep only the rows in which index_al is greater than 200.
4. Inspect the structure and summary statistics of the data.
5. Check for missing values.
6. Replace zero values of index_al and index_rl with missing values and fill them with the column means.
7. Check for duplicate rows.
8. Produce the exploratory visualisations (Chapter 4).
9. Split the data into training and test sets and scale the inputs where required.
10. Train, predict with and evaluate the four regression models.

3.2 Loading and Inspecting the Data

The file is read with *pd.read_csv()*. After the filtering step, the code prints the DataFrame information with *data.info()*, the statistical summary with *data.describe()*, the first and last five rows with *data.head()* and *data.tail()*, the dimensions with *data.shape* and the column names with *data.columns*. The shape of the data is (6531, 7) and the columns are indicator, base_year, year, month, state, index_al and index_rl (Figure 1). The last five rows printed by the code belong to the year 1999-2000, which shows that the file is arranged with the most recent records first and the oldest records at the end (Figure 2).

```
========== DATASET SHAPE ==========
(6531, 7)

========== COLUMN NAMES ==========
Index(['indicator', 'base_year', 'year', 'month', 'state',
'index_al',
       'index_rl'],
      dtype='object')

========== MISSING VALUES ==========
indicator    0
base_year    0
year         0
month        0
state        0
index_al     0
index_rl     0
```

Figure 1: Dataset shape, column names and missing-value counts printed by the project code

```
========== LAST 5 ROWS ==========
          indicator  base_year       year  ...          state
index_al  index_rl
6806  General Index  1986-1987  1999-2000  ...     Tamil Nadu
309.0     308.0
6807  General Index  1986-1987  1999-2000  ...        Tripura
329.0     326.0
6808  General Index  1986-1987  1999-2000  ...  Uttar Pradesh
304.0     304.0
6809  General Index  1986-1987  1999-2000  ...    West Bengal
308.0     309.0
6810  General Index  1986-1987  1999-2000  ...      All India
311.0     311.0
```

Figure 2: Last five rows of the dataset (data.tail())

Table 4 gives the descriptive statistics of the two index columns for the 6,531 retained rows. These values were obtained by applying the describe() function to the retained data. The mean of each column lies above its median (679.07 against 642 for index_al), which suggests a right-skewed distribution that is examined in Section 4.3, and the standard deviation of about 319 index points shows that the index values are widely spread over the 26-year period.

Table 4: Descriptive statistics of the retained data

Statistic	index_al	index_rl
Count	6531	6531
Mean	679.07	681.79
Standard deviation	318.70	319.93
Minimum	274	276
25th percentile	358	359
Median (50%)	642	645
75th percentile	919	926
Maximum	1551	1532

3.3 Dropping the Inflation Columns

The columns *inflation_al* and *inflation_rl* are removed with *data.drop(columns=[...])*. In the supplied file these two columns contain values only for the rows of the base-year 2019 series, which are removed by the next step. For every row that is retained they are empty. Consequently the project works with index levels only and cannot analyse inflation rates from this file.

3.4 Filtering Rows with index_al > 200

The statement *data = data[data["index_al"] > 200].copy()* keeps only those rows in which the Agricultural CPI is greater than 200. In the project code this step is described as the removal of inconsistent data. On inspection of the supplied file, the 280 rows removed by this condition are exactly the rows of the base-year 2019 series, which carry the year labels "2025" (245 rows) and "2026" (35 rows). Their index values lie below about 150, whereas the rows of the 1986-1987 series range from 274 to 1551. The filter therefore separates two index series that are expressed on different bases and cannot be placed on the same numerical scale. It also removes eight rows in which index_al was recorded as zero. After this step every remaining row belongs to the 1986-1987 series, and all later analyses and models are based on that series only.

3.5 Checking Missing Values

Missing values are counted with *data.isnull().sum()*. For the retained data this count is zero for all seven columns (Figure 1), so the dataset has no empty cells in the columns used.

3.6 Handling Zero Values

The next block of code treats a value of zero in index_al or index_rl as a missing value. It converts such zeros to NaN using *data.loc[condition, column] = np.nan* and then fills the NaN entries with the mean of the respective column using *fillna()*. The mean-imputation formula for a column with n valid values $x_1, x_2, \ldots, x_n$ is:

$$\bar{x} = (x_1 + x_2 + \ldots + x_n) / n$$

In the supplied data, the only zero values were the eight rows of index_al that had already been removed by the filter in Section 3.4, and index_rl contained no zeros. The block therefore finds nothing to replace, and no value was actually imputed. The check is harmless, but it did not change the dataset in this project.

3.7 Checking Duplicate Rows

Duplicate records are counted with *data.duplicated().sum()*. Running the check on the retained data returns zero duplicate rows, so no record had to be removed. It should be noted that many different rows share the same pair of index values; this is expected in monthly data from several states and does not make them duplicate records, because the state, year and month differ.

3.8 Train–Test Split and Feature Scaling

For the regression stage the input matrix is X = data["index_rl"] and the target vector is y = data["index_al"]. The data are divided with *train_test_split(X, y, test_size=0.2, random_state=42)*, which places 80% of the rows (5,224) in the training set and 20% (1,307) in the test set. The fixed random_state makes the split reproducible.

Three of the four models receive inputs scaled with *MinMaxScaler*, which rescales a variable to the range 0 to 1 using the formula:

$$x' = (x - x_min) / (x_max - x_min)$$

The scaler is fitted on the training set only and then applied to the test set, so that no information from the test set influences the scaling. Linear Regression receives scaled inputs and a scaled target; KNN Regression and Support Vector Regression receive scaled inputs with the target left unscaled; the Decision Tree uses the unscaled input. Scaling matters for KNN and SVR because these algorithms depend on distances or on the magnitude of the input, whereas a decision tree depends only on the order of the values and is not affected by scaling.

3.9 Summary of Pre-processing

Table 5: Summary of the pre-processing steps implemented in the project

Step	Code used	Effect on the supplied data
Drop inflation columns	data.drop(columns=[...])	9 columns reduced to 7; the dropped columns were empty for all retained rows
Filter rows	data[data["index_al"] > 200]	280 rows of the base-2019 series (including 8 zero values) removed; 6,531 rows kept
Missing-value check	isnull().sum()	Zero missing values
Zero replacement and mean imputation	loc[...] = np.nan; fillna(mean)	No zeros present after filtering; nothing imputed
Duplicate check	duplicated().sum()	Zero duplicate rows
Scaling	MinMaxScaler (fitted on training data)	Applied for Linear Regression, KNN and SVR; not applied for Decision Tree
Train–test split	train_test_split(test_size=0.2, random_state=42)	5,224 training rows and 1,307 test rows

4. Analysis on Dataset

This chapter presents the analyses carried out in the project. Sections 4.1 to 4.7 are exploratory analyses of the Agricultural CPI and its relationship with the Rural CPI. Sections 4.8 to 4.13 describe the regression modelling stage. Every analysis is described under the five headings required by the university format. Figures 4, 8 and 9 and Figures 10 to 13 are reproduced from the output of the project code. Figures 3, 5, 6 and 7 were regenerated by running the plotting code of the project on the supplied dataset, and this is stated in their captions.

4.1 Relationship between Rural CPI and Agricultural CPI

4.1.1 Introduction

This analysis asks whether the Rural CPI and the Agricultural CPI move together and how strong that relationship is. The question is the basis of the whole regression stage: if the two indices were unrelated, no model could estimate one from the other. A scatter plot is the standard first tool for judging the form and strength of a relationship between two numeric variables.

4.1.2 General Description

Each point of the scatter plot represents one record of the dataset, that is, one state in one month of one year. The horizontal position of the point is the Rural CPI of that record and the vertical position is its Agricultural CPI. If the points lie close to a straight line, the two variables have a strong linear relationship; a wide cloud would indicate a weak one.

4.1.3 Specific Requirements, Functions and Formulas

The plot uses the columns *index_rl* and *index_al* of the cleaned DataFrame and is drawn with *sns.scatterplot(x="index_rl", y="index_al", data=data, alpha=0.9)*. The strength of a linear relationship is commonly summarised by the Pearson correlation coefficient:

$$r = \Sigma\,(x_i - \bar{x})(y_i - \bar{y}) \,/\, \sqrt{[\,\Sigma\,(x_i - \bar{x})^2 \cdot \Sigma\,(y_i - \bar{y})^2\,]}$$

where x is the Rural CPI and y is the Agricultural CPI. The value of r lies between −1 and +1. The project code does not print r; the value reported below was calculated separately from the same retained data as a supplementary check.

4.1.4 Analysis Results

The scatter plot (Figure 3) shows the points forming a narrow, upward-sloping band that runs from the lower left corner, where both indices are close to 300, to the upper right corner, where

both indices are above 1500. The points follow an almost straight line over the whole range. The supplementary calculation gives a Pearson correlation of about 0.9993 between the two columns. The average difference between the Rural CPI and the Agricultural CPI of a record is only about 2.7 index points, and the average absolute difference is about 6.7 points; the largest absolute difference in any record is 204 points. Because so many records overlap in the plot, individual deviations from the line are hard to see.

The analysis supports the conclusion that the two indices are very strongly positively correlated in this dataset and that a straight line describes their relationship well. A strong correlation shows association only. The data do not show that the Rural CPI causes the Agricultural CPI to change, and the close agreement of the two series may reflect common underlying price movements, a possibility that the dataset alone cannot confirm.

4.1.5 Visualization

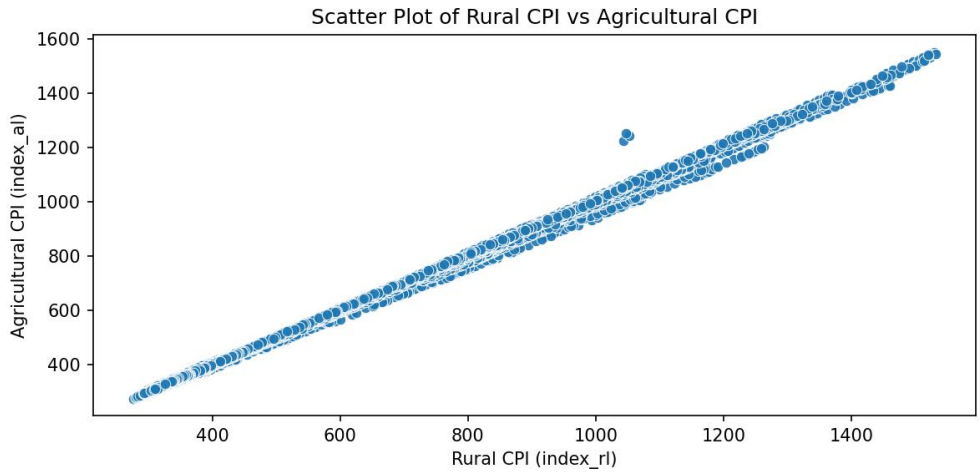


Figure 3: Scatter plot of Rural CPI versus Agricultural CPI (regenerated from the project code and the supplied dataset)

4.2 Top Five States by Maximum Agricultural CPI

4.2.1 Introduction

This analysis asks which states recorded the highest Agricultural CPI values in the dataset. It is useful for seeing which regions reached the highest index levels over the period covered.

4.2.2 General Description

For every state, the highest Agricultural CPI recorded in any month of any year is found. The five states with the largest such maximum values are then compared in a bar chart.

4.2.3 Specific Requirements, Functions and Formulas

The code groups the data by the column *state*, takes the maximum of *index_al* with *groupby("state")["index_al"].max()*, sorts the result in descending order with *sort_values(ascending=False)*, keeps the first five entries with *head(5)* and plots them with *sns.barplot()* using the viridis palette. The quantity plotted for a state s is:

$$\text{Max_s} = \max \{ \text{index_al of all records belonging to state s} \}$$

4.2.4 Analysis Results

Figure 4 shows that Tamil Nadu has the highest maximum Agricultural CPI, followed by Karnataka, Andhra Pradesh, Maharashtra and Haryana. Table 6 gives the exact maximum values. The first two states are very close to each other, with values of about 1551 and 1543, while the maximum then falls to about 1501 for Andhra Pradesh, 1487 for Maharashtra and 1434 for Haryana.

Table 6: Five highest state-wise maximum Agricultural CPI values

Rank	State	Maximum index_al
1	Tamil Nadu	1551
2	Karnataka	1543
3	Andhra Pradesh	1501
4	Maharashtra	1487
5	Haryana	1434

One feature of the chart must be noted. The y-axis is set with *plt.ylim(1400, 1600)*, so it starts at 1400 and not at zero. This makes the differences between the bars look larger than they are; the bar for Haryana is not one third of the bar for Tamil Nadu, but about 8% lower in value. Since the index grows over time, the maximum of a state is expected to occur in the most recent period, so this ranking shows which states reached the highest index levels and not which states have the fastest price rise.

4.2.5 Visualization

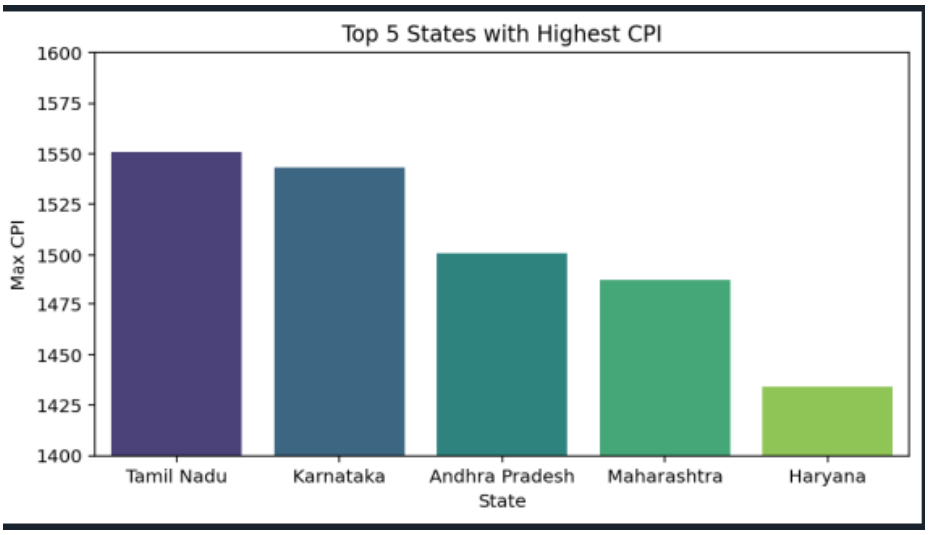


Figure 4: Top five states with the highest maximum Agricultural CPI (output of the project code)

4.3 Distribution of Agricultural CPI

4.3.1 Introduction

This analysis examines how the Agricultural CPI values are distributed across all records: whether they are concentrated in a particular range, symmetric or skewed, and whether they form one group or several.

4.3.2 General Description

A histogram divides the range of a variable into equal-width intervals, called bins, and displays the number of records that fall into each bin. The height of a bar therefore represents the frequency of values in that interval.

4.3.3 Specific Requirements, Functions and Formulas

The histogram is drawn with *plt.hist(data["index_al"], bins=20, color="blue", edgecolor="black", alpha=0.7)*. With 20 bins, the width of each bin is:

$$\text{bin width} = (\max - \min) \,/\, \text{number of bins} = (1551 - 274) \,/\, 20 \approx 63.85$$

4.3.4 Analysis Results

The histogram (Figure 5) is not symmetric. The tallest bar is the first one, covering Agricultural CPI values from 274 to about 338, which contains 1,194 of the 6,531 records (about 18%). The second bar contains 818 records, so the two lowest bins together hold about 31% of all records. The frequency then drops sharply to 239 records in the bin from about 466 to 529, rises again

to a second group of bins between roughly 721 and 912 that hold 406 to 443 records each, and declines gradually towards the right, reaching 25 records in the highest bin (about 1487 to 1551). The long right tail, together with the mean of 679.07 lying above the median of 642, indicates a right-skewed distribution with more than one peak.

The pattern is consistent with the index having risen at different speeds in different periods: when the index moves quickly through a range of values, few monthly records fall in that range, and when it moves slowly, many records accumulate there. The year-wise trend in Section 4.4 is consistent with this explanation, but the histogram alone does not prove it. The distribution should not be treated as a normal distribution.

4.3.5 Visualization

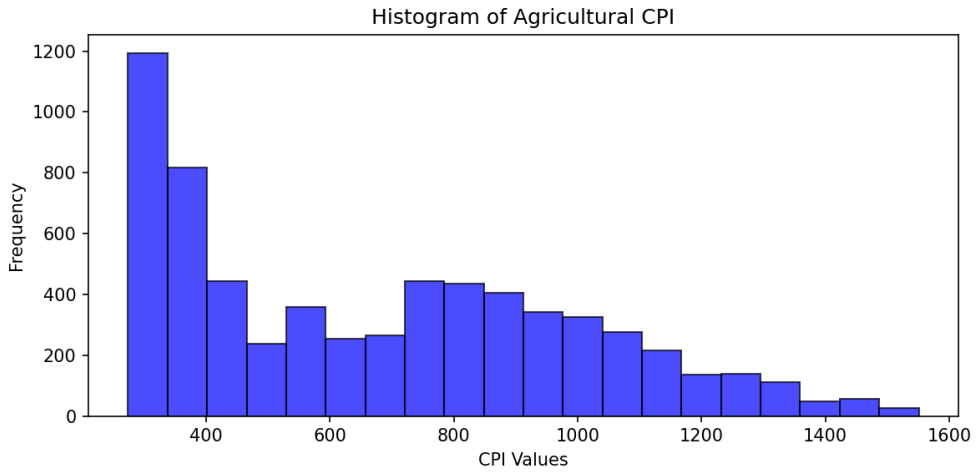


Figure 5: Histogram of Agricultural CPI (regenerated from the project code and the supplied dataset)

4.4 Year-wise Trend of Average Agricultural CPI

4.4.1 Introduction

This analysis asks how the Agricultural CPI changed from year to year. A trend plot condenses the monthly, state-wise records into one value per year so that the long-term direction becomes visible.

4.4.2 General Description

All records belonging to the same year label are averaged, giving one average Agricultural CPI for each of the 26 year labels. The averages are joined by a line with a marker at each year.

4.4.3 Specific Requirements, Functions and Formulas

The code uses *data.groupby("year")["index_al"].mean()* to compute the average and *plt.plot(..., marker="o")* to draw the line. For a year t containing n_t records the plotted value is:

$$\bar{Y}_t = (1 / n_t)\ \Sigma \text{ index_al of the records in year t}$$

4.4.4 Analysis Results

The line in Figure 6 rises from about 312.7 in 1999-2000 to about 1275.4 in 2024-2025. In the first two years the average is almost flat, with a slight dip to about 310.3 in 2000-2001; after that the average increases in every year. The increase is gentle until about 2005-2006, becomes steeper between 2008-2009 and 2013-2014, when the yearly average rises from about 458 to about 754, and is flatter again between about 2015-2016 and 2018-2019 before rising faster in the most recent years. The percentage change of the yearly average, calculated from the plotted values, was about 14% in 2009-2010 and about 10% in 2012-2013 and 2013-2014, compared with about 2% to 4% in 2015-2016 to 2018-2019. These percentages are calculated from the average index levels and are not inflation rates reported in the dataset.

Two cautions apply. First, the average of 2024-2025 is based on 11 months, since June is missing, and may not be exactly comparable with the full-year averages. Second, each year average pools all states, so it describes the typical level and hides differences between states.

4.4.5 Visualization

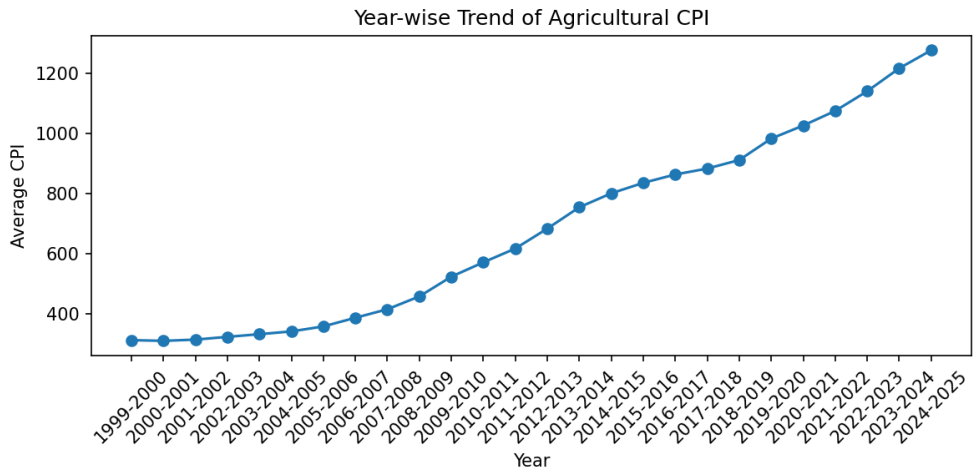


Figure 6: Year-wise trend of average Agricultural CPI (regenerated from the project code and the supplied dataset)

4.5 Three-Year Moving Average of Agricultural CPI

4.5.1 Introduction

This analysis smooths the yearly series of Section 4.4 to show its underlying direction. A moving average reduces short-term variation and is a simple way to emphasise the trend.

4.5.2 General Description

Each value of the moving average is the mean of the yearly average for the current year and for the two preceding years. The first two years have fewer than three values behind them, so no moving average is available for them and the smoothed line starts in the third year.

4.5.3 Specific Requirements, Functions and Formulas

The code applies *year_trend.rolling(3).mean()* to the yearly average series and plots the original series (label "Actual") together with the smoothed series (label "Moving Average"). The formula is:

$$\mathrm{MA}_t = (\bar{Y}_t + \bar{Y}_{t-1} + \bar{Y}_{t-2}) \,/\, 3$$

4.5.4 Analysis Results

In Figure 7 the smoothed line begins at 2001-2002 with a value of about 312.5 and ends at 2024-2025 with a value of about 1210.1, compared with the actual value of about 1275.4 in that year. For example, the final moving average is the mean of the yearly averages of 2022-2023 (about 1139.4), 2023-2024 (about 1215.5) and 2024-2025 (about 1275.4). Because the moving average uses past years, it lies below the actual line whenever the series is rising and therefore lags behind turning points. The smoothed line is steadily upward, which confirms the long-term rise seen in Section 4.4. The three-year moving average is a descriptive tool only; the project does not use it for forecasting.

4.5.5 Visualization

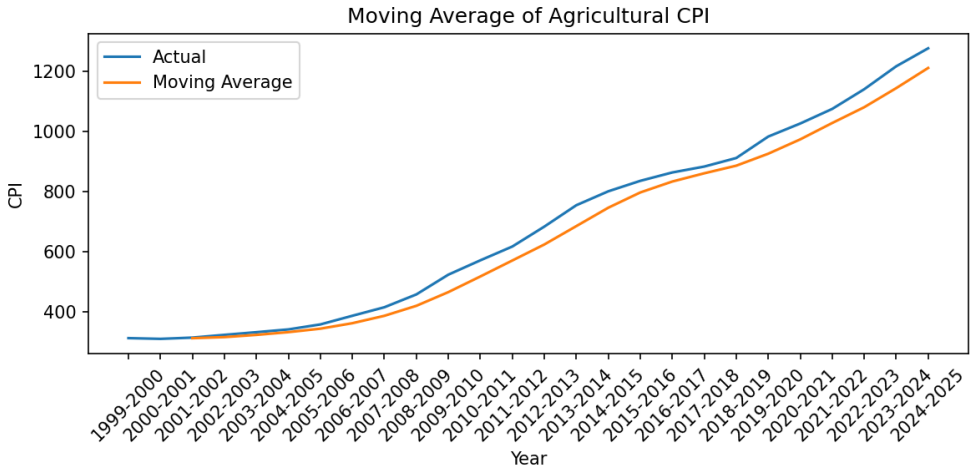


Figure 7: Three-year moving average of Agricultural CPI (regenerated from the project code and the supplied dataset)

4.6 Minimum versus Maximum Agricultural CPI by Year

4.6.1 Introduction

This analysis compares, for each year, the lowest and the highest Agricultural CPI recorded in any state and month. It shows whether the spread of index values across states and months has changed over time.

4.6.2 General Description

Two lines are drawn on the same axes: the first shows the minimum Agricultural CPI of each year and the second shows the maximum. The vertical distance between the lines represents the range of values observed in that year.

4.6.3 Specific Requirements, Functions and Formulas

The code uses *groupby("year")["index_al"].min()* and *groupby("year")["index_al"].max()* and plots the two results as separate lines. The range of a year is:

$$\text{Range}_t = \text{Max}_t - \text{Min}_t$$

4.6.4 Analysis Results

Both lines in Figure 8 rise over the period. The minimum increases from 286 in 1999-2000 to 988 in 2024-2025 and the maximum increases from 347 to 1551. The two lines are close together in the early years, with a range of 61 points in 1999-2000, and move apart progressively; the range is 458 points in 2019-2020 and 563 points in 2024-2025. Table 7 lists

the values. In relative terms, the maximum is about 1.2 times the minimum in 1999-2000 and about 1.6 times the minimum in 2024-2025.

The widening gap shows that the Agricultural CPI levels of different states have drifted apart over the years, so the states with the highest and the lowest index values are further apart now than they were at the start of the period. Since the index measures change relative to a base period, this is a statement about index levels and not about the absolute cost of living in the states. The year 2024-2025 contains only 11 months.

Table 7: Yearly average, minimum, maximum and range of Agricultural CPI

Year	Average	Minimum	Maximum	Range
1999-2000	312.7	286	347	61
2000-2001	310.3	274	350	76
2001-2002	314.4	283	359	76
2002-2003	323.6	291	362	71
2003-2004	332.4	303	356	53
2004-2005	341.7	303	366	63
2005-2006	358.2	322	395	73
2006-2007	387.0	332	429	97
2007-2008	415.2	360	472	112
2008-2009	457.9	385	529	144
2009-2010	523.5	421	615	194
2010-2011	571.1	475	660	185
2011-2012	616.9	492	721	229
2012-2013	683.0	538	808	270
2013-2014	754.1	583	860	277
2014-2015	800.4	648	909	261
2015-2016	835.1	676	993	317
2016-2017	862.9	714	1051	337
2017-2018	882.5	715	1076	361
2018-2019	910.9	726	1095	369
2019-2020	982.1	756	1214	458
2020-2021	1025.5	786	1260	474
2021-2022	1074.0	829	1299	470
2022-2023	1139.4	890	1391	501
2023-2024	1215.5	932	1498	566

Year	Average	Minimum	Maximum	Range
2024-2025	1275.4	988	1551	563

4.6.5 Visualization

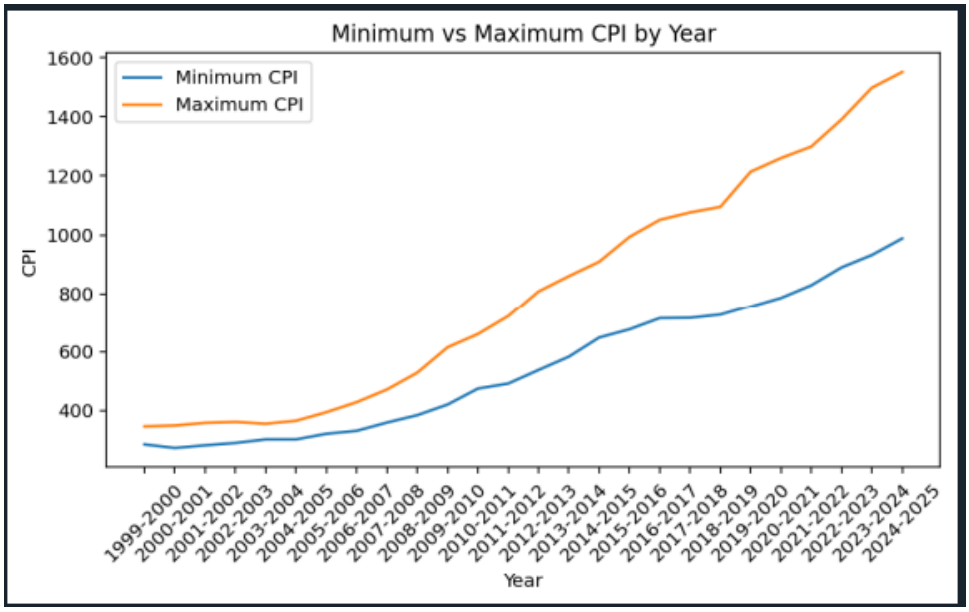


Figure 8: Minimum versus maximum Agricultural CPI by year (output of the project code)

4.7 Top versus Bottom State by Mean Agricultural CPI

4.7.1 Introduction

This analysis identifies the state with the highest and the state with the lowest average Agricultural CPI over the whole period, and compares the two. It answers the project objective of identifying states with high and low Agricultural CPI values.

4.7.2 General Description

The Agricultural CPI of every record of a state is averaged to give a single mean value for that state. The state with the largest mean and the state with the smallest mean are then displayed side by side in a bar chart.

4.7.3 Specific Requirements, Functions and Formulas

The code computes *state_avg = data.groupby("state")["index_al"].mean()*, finds the two states with *idxmax()* and *idxmin()* and draws them with *plt.bar()* in green and red. For a state s with m_s records the mean is:

$$\text{Mean_s} = (1 / m_s)\ \Sigma\ \text{index_al of the records of state s}$$

4.7.4 Analysis Results

Figure 9 shows that Andhra Pradesh has the highest mean Agricultural CPI, about 753.4, and Himachal Pradesh has the lowest, about 569.9; the difference is about 183.5 index points. Unlike the chart in Section 4.2, the y-axis of this chart starts at zero, so the heights of the bars give a fair visual comparison. Table 8 extends the comparison to all 21 entries by applying the same grouped mean; the All-India aggregate has a mean of about 687.6. Both of the two extreme states have 311 records, so their averages are based on the same months. The top group, from Andhra Pradesh to Rajasthan, is tightly clustered between about 730 and 753.

Because every state index is measured against its own base period, a high mean value indicates a larger cumulative rise relative to that base and not a higher absolute price level. The analysis also averages over 26 years, so it hides the changes over time shown in Sections 4.4 and 4.6.

Table 8: Mean Agricultural CPI of all 21 entries, in descending order

Rank	State / region	Mean index_al
1	Andhra Pradesh	753.4
2	Karnataka	751.0
3	Haryana	747.1
4	Tamil Nadu	745.4
5	Maharashtra	743.3
6	Punjab	733.1
7	Rajasthan	730.1
8	Kerala	701.6
9	Gujarat	699.5
10	Meghalaya	697.1
11	All India	687.6
12	Jammu & Kashmir	664.6
13	Assam	659.7
14	Uttar Pradesh	644.9
15	Manipur	644.6
16	Madhya Pradesh	634.1
17	West Bengal	625.9
18	Odisha	617.2
19	Tripura	610.3
20	Bihar	600.3
21	Himachal Pradesh	569.9

4.7.5 Visualization

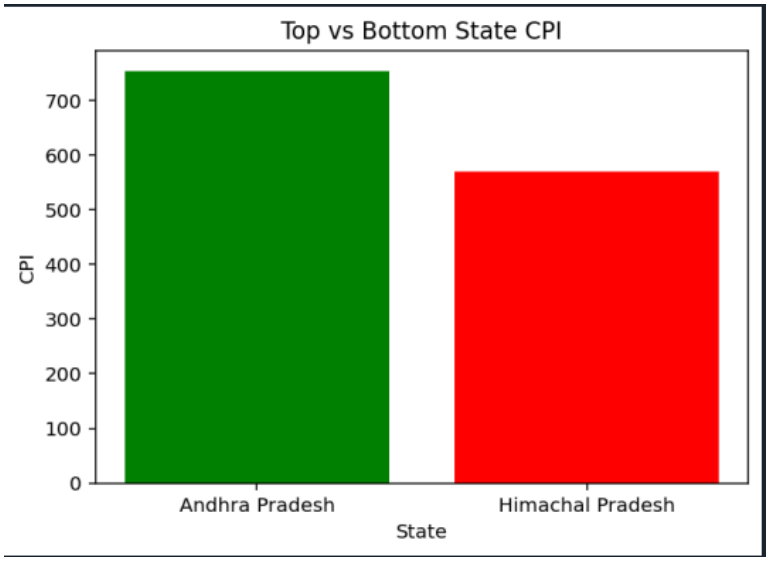


Figure 9: Top versus bottom state by mean Agricultural CPI (output of the project code)

4.8 Regression Set-up and Evaluation Metrics

4.8.1 Introduction

This part of the project asks how accurately the Agricultural CPI of a record can be estimated from the Rural CPI of the same record, and which of four regression algorithms does this best. Regression is the correct type of learning problem because the target, the Agricultural CPI, is a continuous numerical value. Classification measures such as accuracy and the confusion matrix are not meaningful for such a target and are not used.

4.8.2 General Description

In supervised regression a model learns from examples in which both the input and the correct output are known. The data are divided into a training set, from which the model learns, and a test set, which the model has not seen and which is used to measure how well the learned relationship generalises. In this project the input feature is index_rl and the target is index_al. The test set contains 1,307 records, whose Agricultural CPI values range from 285 to 1551 with a mean of about 692.8.

4.8.3 Specific Requirements, Functions and Formulas

The split is produced by *train_test_split(X, y, test_size=0.2, random_state=42)*. Every model is trained on the same 5,224 training records and evaluated on the same 1,307 test records, so

that the scores are directly comparable. Table 9 lists the configuration of the four models as set in the code.

Table 9: Configuration of the four regression models

Model	Scikit-learn class	Settings in the project	Input scaling
Linear Regression	LinearRegression()	Default settings	MinMaxScaler on input and target
Decision Tree	DecisionTreeRegressor	max_depth=5, min_samples_split=10, min_samples_leaf=5, random_state=10	None
KNN Regression	KNeighborsRegressor	n_neighbors=21	MinMaxScaler on input
Support Vector Regression	SVR	kernel="linear", C=1000	MinMaxScaler on input

The models are compared with three metrics, each computed with the Scikit-learn functions *mean_absolute_error*, *mean_squared_error* and *r2_score*. Let y_i be the actual Agricultural CPI of test record i, $\hat{y}_i$ the predicted value, $\bar{y}$ the mean of the actual values and n the number of test records (1,307). The **Mean Absolute Error** is the average size of the errors, expressed in index points:

$$\mathrm{MAE} = (1/n)\ \Sigma\ |y_i - \hat{y}_i|$$

The **Mean Squared Error** is the average of the squared errors. Squaring gives larger errors a greater weight, so MSE is more sensitive than MAE to occasional large mistakes, and its unit is the square of the index points:

$$\mathrm{MSE} = (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2$$

The **R^2 score** (coefficient of determination) compares the model with a baseline that always predicts the mean:

$$R^2 = 1 - \Sigma\ (y_i - \hat{y}_i)^2\ /\ \Sigma\ (y_i - \bar{y})^2$$

An R^2 of 1 means perfect prediction and a value near 0 means the model is no better than predicting the mean. For MAE and MSE, lower values are better; for R^2, higher values are better. Since all models are evaluated on the same test records, the denominator of R^2 is the same for all models, and the ranking of the models by R^2 is therefore the reverse of their ranking by MSE.

4.8.4 Analysis Results

The split gives 5,224 training and 1,307 test records. The mean Agricultural CPI of the training set is about 675.6, and that of the test set is about 692.8. The detailed results of each model follow in Sections 4.9 to 4.12 and are compared in Section 4.13.

4.8.5 Visualization

This section has no figure of its own; the console outputs and plots of the individual models appear in the sections that follow.

4.9 Linear Regression

4.9.1 Introduction

Linear Regression is the simplest regression model and serves as the reference model of the project. This analysis asks how well a single straight line explains the Agricultural CPI as a function of the Rural CPI, a question suggested by the nearly straight scatter plot of Section 4.1.

4.9.2 General Description

Linear Regression assumes that the target is a linear function of the input. With one input feature the model is a straight line with an intercept β_0 and a slope β_1 [7]. It is appropriate for regression because it outputs a continuous value for any input. The parameters are found by the method of least squares, which chooses the line that minimises the sum of the squared differences between the actual and predicted values of the training data. To predict, the learned line is evaluated at the input value; to evaluate, the predictions for the test set are compared with the actual values using the metrics of Section 4.8.

4.9.3 Specific Requirements, Functions and Formulas

In the project the training input is scaled with a MinMaxScaler and the training target is scaled with a second MinMaxScaler. The model is created with *LinearRegression()* and trained with *model.fit(x_train_scaled, y_train_scaled)*. Predictions on the test set are computed with *model.predict()* and converted back to index points with *y_scaler.inverse_transform()* before the metrics are calculated, so that MAE and MSE are expressed in the original units. The model and the least-squares solution are:

$$\hat{y} = \beta_0 + \beta_1 x, \quad \beta_1 = \Sigma\,(x_i - \bar{x})(y_i - \bar{y}) \,/\, \Sigma\,(x_i - \bar{x})^2, \quad \beta_0 = \bar{y} - \beta_1 \bar{x}$$

4.9.4 Analysis Results

The project also tests a normalised Rural CPI input of 0.5. This corresponds to a Rural CPI of 900.5 (the midpoint of the training range, 276 to 1525), and the model predicts an Agricultural CPI of 897.01 (Figure 10). Re-fitting the same model on the same training data gives a slope of about 0.978 and an intercept of about 0.0008 in the scaled space, which is equivalent to $\hat{y} \approx 0.215 + 0.9959 \times \text{index_rl}$ in the original units. These coefficient values are not printed by the project code; they were obtained by re-fitting the model. A slope this close to 1 and an intercept this close to 0 mean that the fitted line is almost the line y = x, so the Agricultural CPI is estimated to be nearly equal to the Rural CPI.

On the test set, Linear Regression achieves an MAE of 7.2751, an MSE of 152.3308 and an R^2 of 0.9985. The MAE of 7.28 index points is about 1.05% of the mean test value of 692.8. The first test predictions are shown in Table 11 in Section 4.13.

4.9.5 Visualization

The project does not draw a plot for Linear Regression. Figure 10 shows the console output of the project for Linear Regression, Decision Tree and KNN Regression, including the prediction for the input 0.5 and the first five test predictions.

```
========== LINEAR REGRESSION ==========
Normalized Rural CPI input: 0.5
Predicted Agricultural CPI: 897.01
First 5 test predictions: [ 967.21634547  913.43870066 1096.68104594
304.95460843 1177.34751316]

========== DECISION TREE REGRESSION ==========
Normalized Rural CPI input: 0.5
Predicted Agricultural CPI: 910.72
First 5 test predictions: [ 953.62857143  910.72486772 1084.97692308
302.40379404 1188.41860465]

========== KNN REGRESSION ==========
Normalized Rural CPI input: 0.5
Predicted Agricultural CPI: 897.9
First 5 test predictions: [ 963.19047619  905.14285714 1093.42857143
305.0952381  1181.         ]
```

Figure 10: Console output for Linear Regression, Decision Tree and KNN Regression

4.10 Decision Tree Regression

4.10.1 Introduction

This analysis asks whether a non-linear, rule-based model can estimate the Agricultural CPI better than the straight line. A decision tree makes no assumption of linearity and is easy to interpret, since its decisions can be read as a series of questions.

4.10.2 General Description

A regression tree repeatedly divides the range of the input into two parts by choosing a threshold value of the feature, a procedure known as CART [8]. At each node, the threshold is chosen so that the squared differences between the training targets and the mean target of the two resulting groups are as small as possible. The process continues until a stopping rule is met. The final groups are called leaves, and the prediction for any input falling into a leaf is the mean of the training targets in that leaf. A tree is appropriate for regression because every leaf holds a numerical value. The predictions form a step-like function, which means that the model cannot predict values outside the range of the targets it has seen.

4.10.3 Specific Requirements, Functions and Formulas

The model is created with *DecisionTreeRegressor(max_depth=5, min_samples_split=10, min_samples_leaf=5, random_state=10)* and trained on the unscaled training data with *dt_model.fit(X_train, y_train)*. The restrictions mean that the tree has at most five levels of splits, that a node must hold at least 10 records to be divided, and that each leaf must contain at least 5 records; they limit the size of the tree and reduce the risk that it memorises the training data. The split of a node is chosen to minimise:

$$\Sigma\,(y_i - \bar{y}_\text{left})^2 + \Sigma\,(y_i - \bar{y}_\text{right})^2$$

and the prediction of a leaf containing the training records L is $\hat{y} = (1/|L|)\,\Sigma\, y_i$ over the records of L. Predictions on the test set are obtained with *dt_model.predict(X_test)* and are scored with the metrics of Section 4.8.

4.10.4 Analysis Results

For the input 0.5 (Rural CPI 900.5) the tree predicts 910.72, compared with 897.01 for Linear Regression. Re-fitting the tree shows that it reaches the maximum depth of 5 and has 32 leaves, so it can output at most 32 distinct values. The first split divides the training records at a Rural CPI of 698.5; the next splits, at about 463.5 on the lower side and 1025.5 on the upper side, are visible in Figure 11. The root node contains all 5,224 training records, and the mean Agricultural CPI of these records is about 675.6. The root threshold was read from the re-fitted tree, because the printed figure is too small to read reliably.

On the test set the Decision Tree obtains an MAE of 11.6116, an MSE of 280.3189 and an R^2 of 0.9973. It is the weakest of the four models on all three metrics: its MAE is about 60% higher than that of Linear Regression, and its MSE is about 1.8 times as large. A plausible reason is that a tree with at most 32 leaves approximates a nearly straight relationship with steps, so that

records in the same leaf receive the same prediction although their actual values differ. This explanation matches the results but was not tested separately.

4.10.5 Visualization

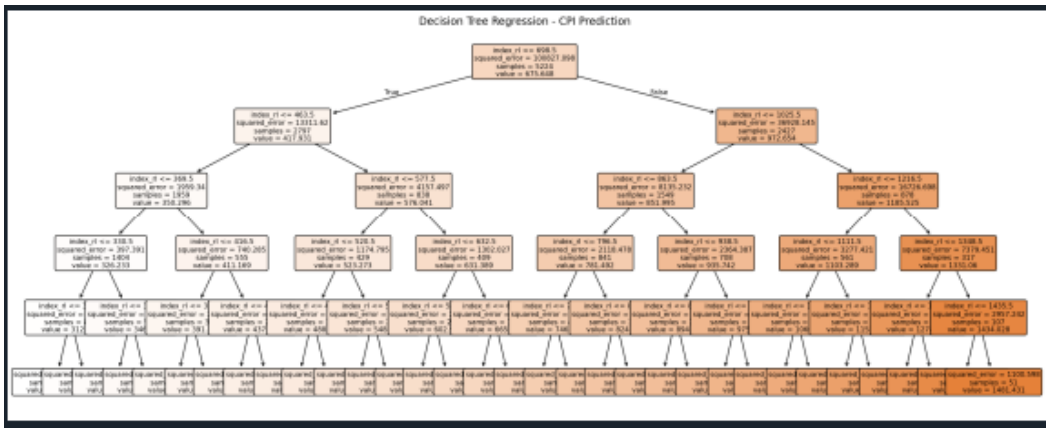


Figure 11: Decision Tree Regression plot produced by plot_tree() (output of the project code)

The leaf nodes of the figure are small because the original plot was captured at a low resolution; a higher-resolution export of the same plot can be substituted without changing the analysis.

4.11 K-Nearest Neighbors (KNN) Regression

4.11.1 Introduction

This analysis asks whether estimating the Agricultural CPI directly from similar records, without fitting any equation, performs as well as the other models. KNN is a simple, instance-based method that is commonly used as a reference for non-parametric regression.

4.11.2 General Description

KNN Regression stores the training data and does no further learning during training [9]. To predict for a new input, it finds the k training records whose inputs are closest to the new input and returns the average of their target values. It is suitable for regression because averaging numerical targets gives a continuous estimate. The distance between records is measured on the scaled inputs, so that the result does not depend on the units of the feature. With one feature, the nearest neighbours are simply the records whose Rural CPI is closest to the given value.

4.11.3 Specific Requirements, Functions and Formulas

The input is scaled with a MinMaxScaler fitted on the training data. The model is created with *KNeighborsRegressor(n_neighbors=21)*, trained with *knn_model.fit(X_train_knn, y_train)* and used with *knn_model.predict(X_test_knn)*. The default Scikit-learn settings apply for all

other parameters, which means that the 21 neighbours are given equal weight and that distance is the Euclidean distance. The prediction and the distance are:

$$\hat{y} = (1/k)\,\Sigma\, y_i \text{ over the k nearest neighbours}, \quad d(x, x') = \sqrt{\Sigma\,(x_j - x'_j)^2}$$

with k = 21. The value of k is a setting chosen by the author; the project does not show a tuning procedure for it.

4.11.4 Analysis Results

For the input 0.5 the KNN model predicts 897.9, which is close to the 897.01 of Linear Regression (Figure 10). On the test set KNN achieves an MAE of 7.5307, an MSE of 158.5076 and an R^2 of 0.9985. Its error is slightly larger than that of Linear Regression and Support Vector Regression and much smaller than that of the Decision Tree. Because several training records share the same Rural CPI value and the model averages a fixed number of neighbours, ties between equally distant records may be resolved differently in different software environments, so KNN scores can differ slightly when the code is run on another machine.

4.11.5 Visualization

The project does not produce a separate plot for KNN. The prediction for the input 0.5 and the first test predictions of the model are shown in Figure 10 in Section 4.9.

4.12 Support Vector Regression (SVR)

4.12.1 Introduction

This analysis asks how a margin-based model performs on the same task. Support Vector Regression is a widely used algorithm that tries to fit the data within a tolerance band while keeping the model simple.

4.12.2 General Description

Support Vector Regression fits a function so that most training points lie within a tolerance ε of the predictions; errors smaller than ε are not penalised [10]. Points outside this band are penalised, and a parameter C controls how heavily. A large C places a high penalty on points outside the band and makes the fit follow the training data closely. SVR can use different kernels; a linear kernel gives a straight-line function, whereas other kernels such as the radial basis function produce curves. The code contains the comment "Changed to linear kernel", showing that the author selected the linear kernel deliberately. SVR is appropriate for regression because it outputs a continuous value for any input.

4.12.3 Specific Requirements, Functions and Formulas

The input is scaled with a MinMaxScaler, while the target is left unscaled. The model is created with *SVR(kernel="linear", C=1000)*, trained with *svr_model.fit(X_train_svr, y_train)* and used with *svr_model.predict(X_test_svr)*. The parameter ε is not set in the code and takes the default Scikit-learn value of 0.1. The training problem, with slack variables ξ_i and ξ_i^* for points outside the band, is:

$$\text{minimise } \tfrac{1}{2}\|w\|^2 + C \sum (\xi_i + \xi_i^*) \quad \text{subject to} \quad |y_i - (w \cdot x_i + b)| \leq \varepsilon + \xi_i$$

For a linear kernel the prediction is $\hat{y} = w \cdot x + b$. The loss that SVR minimises is not the squared error, which is one reason why its ranking may differ between MAE and MSE.

4.12.4 Analysis Results

For the input 0.5 the SVR model predicts 899.49 (Figure 13). On the test set it obtains an MAE of 7.1464, an MSE of 160.5191 and an R^2 of 0.9985. SVR has the lowest MAE of the four models but the third-lowest MSE. The difference between the two metrics suggests that its errors include a few relatively larger deviations, which MSE penalises more than MAE; this interpretation is consistent with the numbers but was not checked against the individual errors. Figure 12 plots the actual and the predicted Agricultural CPI of the test records against the Rural CPI. The predicted values (orange) form a straight line, as expected for a linear kernel and a single input, and the actual values (blue) lie along this line with small scatter around it; some blue points appear slightly below the line over the middle of the range. The orange points cover the blue points almost entirely, so the plot shows a close fit but does not show individual errors precisely.

4.12.5 Visualization

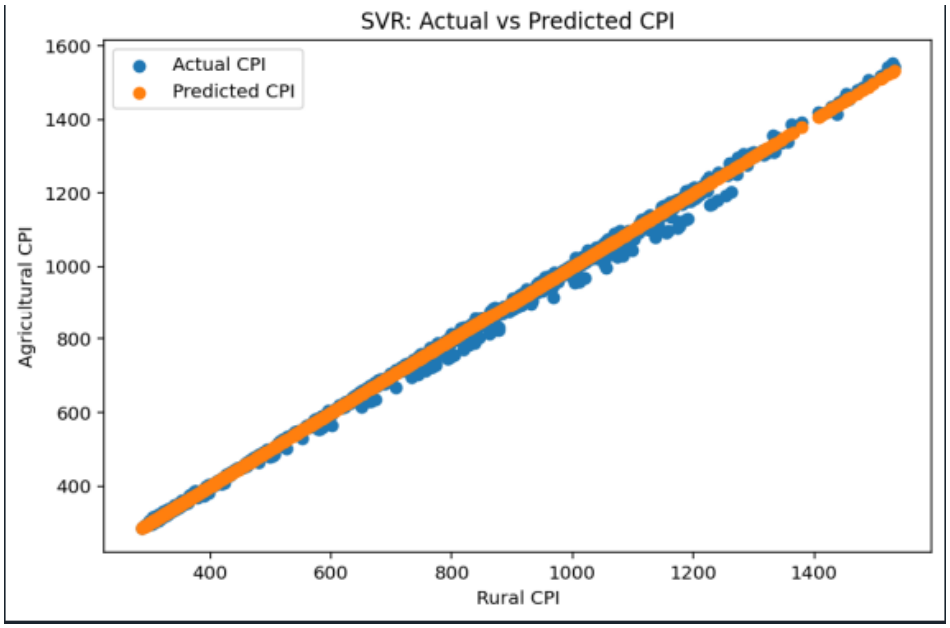


Figure 12: SVR: actual versus predicted Agricultural CPI for the test set (output of the project code)

```
========== SUPPORT VECTOR REGRESSION ==========
Normalized Rural CPI input: 0.5
Predicted Agricultural CPI: 899.49
First 5 test predictions: [ 969.97996931  915.98998686 1099.955853
805.10333354 1180.94082668]

========== FINAL MODEL COMPARISON ==========
                   Model     MAE      MSE  R2 Score
       Linear Regression  7.2751 152.3308    0.9985
 Decision Tree Regression 11.6116 280.3189    0.9973
          KNN Regression  7.5307 158.5076    0.9985
Support Vector Regression  7.1464 160.5191    0.9985

Best model according to R2 Score:
Linear Regression
```

Figure 13: Console output for Support Vector Regression and the final model comparison

4.13 Comparison of the Four Models

4.13.1 Introduction

This analysis answers the main question of the modelling stage: how do the four regression models compare when they are trained and tested on the same data?

4.13.2 General Description

The test-set predictions of each model are compared with the actual values using MAE, MSE and R^2. The project collects the three metrics for the four models in a single table with

pd.DataFrame() and prints it rounded to four decimal places. The same section of code prints the model with the highest R^2 score by using *idxmax()*.

4.13.3 Specific Requirements, Functions and Formulas

The functions *mean_absolute_error()*, *mean_squared_error()* and *r2_score()* of Scikit-learn are applied to the test targets and each model's test predictions, as defined in Section 4.8. For easier interpretation, two quantities were additionally derived from the reported values: the root mean squared error, RMSE = √MSE, which is in index points like the MAE, and the MAE as a percentage of the mean test-set value of 692.8.

4.13.4 Analysis Results

Table 10 lists the Agricultural CPI predicted by each model for the normalised Rural CPI input 0.5, which corresponds to a Rural CPI of 900.5. The four predictions lie between 897.01 and 910.72.

Table 10: Predictions for the normalised Rural CPI input 0.5 (Rural CPI = 900.5)

Model	Predicted Agricultural CPI
Linear Regression	897.01
Decision Tree Regression	910.72
KNN Regression	897.9
Support Vector Regression	899.49

Table 11 compares the first four test predictions of each model with the actual values (the fifth values of the printed arrays are truncated for the KNN model in the available output).

Table 11: First four test predictions compared with the actual Agricultural CPI

Test record	1	2	3	4
Rural CPI (input)	971	917	1101	306
Actual index_al	977	908	1074	305
Linear Regression	967.22	913.44	1096.68	304.95
Decision Tree	953.63	910.72	1084.98	302.40
KNN Regression	963.19	905.14	1093.43	305.10
Support Vector Regression	969.98	915.99	1099.96	305.10

The final comparison of the four models, taken from the output of the project code, is given in Table 12.

Table 12: Final comparison of the four regression models on the test set

Model	MAE	MSE	R^2 Score
Linear Regression	7.2751	152.3308	0.9985
Decision Tree Regression	11.6116	280.3189	0.9973
KNN Regression	7.5307	158.5076	0.9985
Support Vector Regression	7.1464	160.5191	0.9985

Ranked by MAE, Support Vector Regression is first (7.1464), followed by Linear Regression (7.2751), KNN Regression (7.5307) and the Decision Tree (11.6116). Ranked by MSE, and therefore by R^2, Linear Regression is first (152.3308), followed by KNN Regression (158.5076), Support Vector Regression (160.5191) and the Decision Tree (280.3189). The project code reports Linear Regression as the best model according to the R^2 score, which agrees with its position in this ranking. The R^2 values of Linear Regression, KNN Regression and Support Vector Regression are all 0.9985 when rounded to four decimal places, so their differences appear only in the unrounded values and in MAE and MSE.

Table 13: Quantities derived from the reported metrics

Model	RMSE (√MSE)	MAE as % of mean test value
Linear Regression	12.34	1.05%
Decision Tree Regression	16.74	1.68%
KNN Regression	12.59	1.09%
Support Vector Regression	12.67	1.03%

The three best models differ very little: their MAE values lie within about 0.4 index points of each other and their MSE values within about 8 units, whereas the Decision Tree is clearly behind, with an MSE about 1.8 times that of Linear Regression. The R^2 values of all four models are above 0.997, which means that each model explains more than 99.7% of the variation of the Agricultural CPI in the test set. This high level of accuracy is expected from the almost straight relationship found in Section 4.1; it shows that the two indices are very close to each other in this dataset, not that Agricultural CPI can be forecast for future months.

From these results it can be stated that Linear Regression gives the lowest MSE and the highest R^2, and SVR gives the lowest MAE, while the performance of Linear Regression, KNN Regression and SVR is practically equivalent for this dataset. The evidence from one train–test

split is not enough to declare any of the three a clear winner. Linear Regression is the simplest of them and has the advantage of an easily interpretable equation.

4.13.5 Visualization

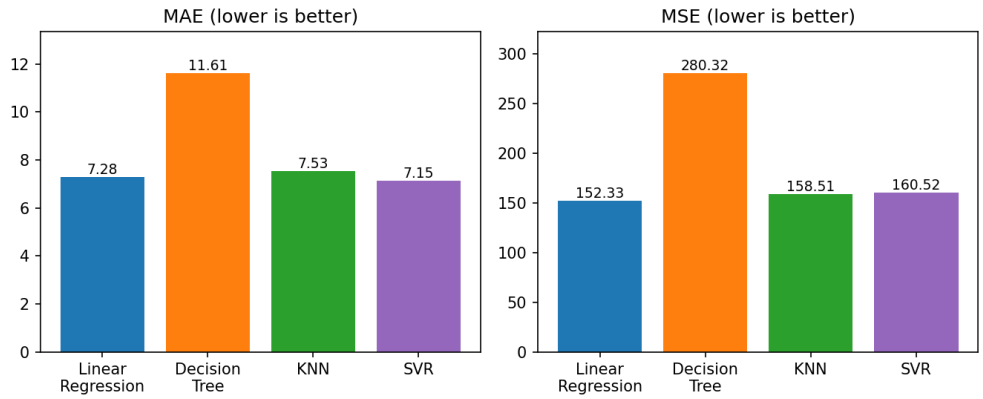


Figure 14: MAE and MSE of the four models (plotted from the results in Table 12)

5. Conclusion

5.1 Summary of Findings

The project analysed the state-wise monthly Consumer Price Index data of the 1986-1987 series from 1999-2000 to 2024-2025 and trained four regression models to estimate the Agricultural CPI from the Rural CPI. The main findings supported by the results are listed below.

- After removing the base-year 2019 rows, the dataset contained 6,531 records with no missing values and no duplicate rows.
- The Rural CPI and the Agricultural CPI are extremely closely related. Their scatter plot is almost a straight line, and a supplementary calculation gave a correlation of about 0.9993. This is an association and does not establish a causal relationship.
- Tamil Nadu, Karnataka, Andhra Pradesh, Maharashtra and Haryana recorded the highest maximum Agricultural CPI values, with Tamil Nadu at 1551. By mean value over the whole period, Andhra Pradesh was highest (about 753.4) and Himachal Pradesh was lowest (about 569.9).
- The distribution of the Agricultural CPI is right-skewed with more than one peak, and the yearly average rose from about 313 in 1999-2000 to about 1275 in 2024-2025, with the fastest rises around 2009-2014.
- The gap between the highest and the lowest Agricultural CPI in a year widened from 61 index points in 1999-2000 to 563 points in 2024-2025.
- On the test set, Linear Regression, KNN Regression and Support Vector Regression achieved similar results ($R^2 = 0.9985$ when rounded), with Linear Regression having the lowest MSE and SVR the lowest MAE. The Decision Tree performed less well ($R^2 = 0.9973$).

5.2 Limitations and Methodological Observations

The following observations should be kept in mind when using the results of this project.

- **Row filter.** The condition index_al > 200 is described in the code as removing inconsistent data. In the supplied file, it removes the base-year 2019 series, which is on a different scale. This is a reasonable separation of two series, but it means the analysis covers the 1986-1987 series only and does not include the 2025 and 2026 records.

- **Zero handling.** The replacement of zeros by column means had no effect in this dataset, because the only zeros had already been removed by the filter. The step cannot be counted as a cleaning result.
- **Inflation columns.** The inflation columns were empty for the retained rows, so the study is limited to index levels. Year-to-year changes quoted in this report are calculated from average index levels and are not official inflation rates.
- **Partial year.** The year 2024-2025 contains 11 months (June is absent), which affects its average, minimum and maximum slightly.
- **Chart scaling.** The y-axis of the top-five chart starts at 1400 and exaggerates the visual differences between states.
- **Single split and tuning.** All scores come from one random 80:20 split. No cross-validation was carried out, and the settings of the models (maximum depth 5, k = 21, C = 1000) were not shown to be the result of systematic tuning.
- **Dependence between records.** The records of different states in the same month are likely to be similar, and a random split can place such similar records in both the training and the test set. This may make the test scores look better than they would for a different kind of split, for example one based on time.
- **Near-identity of the variables.** With a correlation of about 0.9993 between the two indices, the estimation task is easy, and the high R^2 values mainly reflect this closeness. The models do not forecast future CPI.
- **Scaling of the target.** In Linear Regression, scaling the target is not required for the fit, since a least-squares line is unaffected by a linear rescaling; the step is harmless but unnecessary.
- **Reproducibility of KNN.** Because of many tied Rural CPI values, the KNN predictions may differ slightly between software versions. The scores reported here are those printed by the author's run.
- **Interpretation of index values.** Comparison of CPI levels between states shows cumulative change since the base period and not differences in absolute prices.

6. Future Scope

The work can be extended in several directions without changing its basic structure.

- **More input features.** Month, year and state can be added as inputs (state encoded numerically), so that the models can use more information than the Rural CPI alone. Multiple regression and tree-based ensemble methods such as Random Forest or Gradient Boosting can then be compared with the present models.
- **Time-based evaluation.** The data can be divided chronologically, using the earlier years for training and the later years for testing, so that the evaluation reflects the situation in which past data are used to estimate later values.
- **Cross-validation and tuning.** K-fold cross-validation and a systematic search of settings such as the tree depth, the number of neighbours and the SVR parameters C and ε (for example with GridSearchCV) would give more dependable comparisons of the models.
- **Time-series forecasting.** Genuine forecasting of future index values would require time-series methods, for example ARIMA-type models or models with lagged values of the index, and an evaluation designed for forecasting.
- **Additional metrics and diagnostics.** The root mean squared error, residual plots and the distribution of errors by state and by year would show where the models make their largest errors.
- **Inflation analysis.** If the inflation columns for the older series are obtained or derived from the index values, the project can be extended to the analysis of inflation rates and their relation to the index levels.
- **Interactive presentation.** The analyses can be placed in a dashboard that allows a user to select a state and a period.

Appendix A: Python Source Code

The complete source code of the project is reproduced below, without modification, in a fixed-width font so that its layout is preserved.

```
# PROJECT: CPI DATA ANALYSIS AND PREDICTION

# Importing libraries
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.svm import SVR
from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import MinMaxScaler



# Load the dataset
data = pd.read_csv("C:/Users/spars/Downloads/cpialrl_5.csv")

data = data.drop(columns=['inflation_al', 'inflation_rl'])

# Removing inconsistent data
data = data[data['index_al'] > 200].copy()



# DATA SUMMARY
print("\n========== DATASET INFORMATION ==========")
data.info()

print("\n========== STATISTICAL SUMMARY ==========")
print(data.describe())

print("\n========== FIRST 5 ROWS ==========")
print(data.head())

print("\n========== LAST 5 ROWS ==========")
print(data.tail())

print("\n========== DATASET SHAPE ==========")
print(data.shape)

print("\n========== COLUMN NAMES ==========")
print(data.columns)



# CHECKING MISSING VALUES
print("\n========== MISSING VALUES ==========")
print(data.isnull().sum())



# HANDLING MISSING VALUES
data.loc[data["index_al"] == 0, "index_al"] = np.nan
data.loc[data["index_rl"] == 0, "index_rl"] = np.nan
```

```
data['index_al'] = data['index_al'].fillna(data['index_al'].mean())
data['index_rl'] = data['index_rl'].fillna(data['index_rl'].mean())

print("\n========== MISSING VALUES AFTER CLEANING ==========")
print(data.isnull().sum())


# CHECKING DUPLICATE ROWS
print("Duplicate rows:", data.duplicated().sum())


# RELATIONSHIP BETWEEN RURAL CPI AND AGRICULTURAL CPI
plt.figure(figsize=(8, 4))
sns.scatterplot(x='index_rl', y='index_al', data=data, alpha=0.9)
plt.title('Scatter Plot of Rural CPI vs Agricultural CPI')
plt.xlabel('Rural CPI (index_rl)')
plt.ylabel('Agricultural CPI (index_al)')
plt.show()


# TOP 5 STATES WITH HIGHEST CPI
top_states = (
    data.groupby('state')['index_al']
    .max()
    .sort_values(ascending=False)
    .head(5)
    .reset_index()
)

plt.figure(figsize=(8, 4))
sns.barplot(
    x='state',
    y='index_al',
    data=top_states,
    errorbar=None,
    hue='state',
    legend=False,
    palette='viridis'
)
plt.title('Top 5 States with Highest CPI')
plt.xlabel('State')
plt.ylabel('Max CPI')
plt.ylim(1400, 1600)
plt.show()


# DISTRIBUTION OF AGRICULTURAL CPI VALUES
plt.figure(figsize=(8, 4))
plt.hist(
    data['index_al'],
    bins=20,
    color='blue',
    edgecolor='black',
    alpha=0.7
)
plt.title('Histogram of Agricultural CPI')
plt.xlabel('CPI Values')
plt.ylabel('Frequency')
plt.show()


# YEAR-WISE TREND OF AGRICULTURAL CPI
```

```
plt.figure(figsize=(8, 4))
year_trend = data.groupby('year')['index_al'].mean()

plt.plot(year_trend.index, year_trend.values, marker='o')
plt.title('Year-wise Trend of Agricultural CPI')
plt.xlabel('Year')
plt.ylabel('Average CPI')
plt.xticks(rotation=45)
plt.show()



# MOVING AVERAGE TREND OF AGRICULTURAL CPI
plt.figure(figsize=(8, 4))
year_trend = data.groupby('year')['index_al'].mean()
moving_avg = year_trend.rolling(3).mean()

plt.plot(year_trend.index, year_trend.values, label='Actual')
plt.plot(year_trend.index, moving_avg.values, label='Moving Average')

plt.title('Moving Average of Agricultural CPI')
plt.xlabel('Year')
plt.ylabel('CPI')
plt.xticks(rotation=45)
plt.legend()
plt.show()



# MINIMUM VS MAXIMUM CPI BY YEAR
plt.figure(figsize=(8, 4))
year_min = data.groupby('year')['index_al'].min()
year_max = data.groupby('year')['index_al'].max()

plt.plot(year_min.index, year_min.values, label='Minimum CPI')
plt.plot(year_max.index, year_max.values, label='Maximum CPI')

plt.title('Minimum vs Maximum CPI by Year')
plt.xlabel('Year')
plt.ylabel('CPI')
plt.xticks(rotation=45)
plt.legend()
plt.show()



# TOP VS BOTTOM STATE CPI USING MEAN VALUE
plt.figure(figsize=(6, 4))
state_avg = data.groupby('state')['index_al'].mean()

top_state = state_avg.idxmax()
bottom_state = state_avg.idxmin()

values = [state_avg.max(), state_avg.min()]
labels = [top_state, bottom_state]

plt.bar(labels, values, color=['green', 'red'])
plt.title('Top vs Bottom State CPI')
plt.xlabel('State')
plt.ylabel('CPI')
plt.show()



# PREPARING DATA FOR ALL MODELS
X = data[['index_rl']]
```

```
y = data['index_al']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)



# LINEAR REGRESSION

x_scaler = MinMaxScaler()
y_scaler = MinMaxScaler()

x_train_scaled = x_scaler.fit_transform(X_train)
x_test_scaled = x_scaler.transform(X_test)

y_train_scaled = y_scaler.fit_transform(
    y_train.to_frame()
).ravel()

model = LinearRegression()
model.fit(x_train_scaled, y_train_scaled)

# Prediction for normalized Rural CPI
input_value = 0.5

check_value = pd.DataFrame({
    'index_rl': x_scaler.inverse_transform([[input_value]])[0]
})

check_value_scaled = x_scaler.transform(check_value)

result_scaled = model.predict(check_value_scaled)
result = y_scaler.inverse_transform(
    result_scaled.reshape(-1, 1)
).ravel()

print("\n========== LINEAR REGRESSION ==========")
print("Normalized Rural CPI input:", input_value)
print("Predicted Agricultural CPI:", round(result[0], 2))

# Test predictions
y_pred_scaled = model.predict(x_test_scaled)
lr_pred = y_scaler.inverse_transform(
    y_pred_scaled.reshape(-1, 1)
).ravel()

print("First 5 test predictions:", lr_pred[:5])



# DECISION TREE REGRESSION

dt_model = DecisionTreeRegressor(
    max_depth=5,
    min_samples_split=10,
    min_samples_leaf=5,
    random_state=10
)

dt_model.fit(X_train, y_train)
dt_pred = dt_model.predict(X_test)
```

```
# Prediction using the same input
dt_result = dt_model.predict(check_value)

print("\n========== DECISION TREE REGRESSION ==========")
print("Normalized Rural CPI input:", input_value)
print("Predicted Agricultural CPI:", round(dt_result[0], 2))
print("First 5 test predictions:", dt_pred[:5])

# Decision Tree plot
plt.figure(figsize=(20, 8))

plot_tree(
    dt_model,
    feature_names=['index_rl'],
    filled=True,
    rounded=True,
    fontsize=8
)

plt.title("Decision Tree Regression - CPI Prediction")
plt.show()



# KNN REGRESSION

knn_scaler = MinMaxScaler()

X_train_knn = knn_scaler.fit_transform(X_train)
X_test_knn = knn_scaler.transform(X_test)

knn_model = KNeighborsRegressor(n_neighbors=21)
knn_model.fit(X_train_knn, y_train)

knn_pred = knn_model.predict(X_test_knn)

# Prediction using the same input
check_value_scaled = knn_scaler.transform(check_value)
knn_result = knn_model.predict(check_value_scaled)

print("\n========== KNN REGRESSION ==========")
print("Normalized Rural CPI input:", input_value)
print("Predicted Agricultural CPI:", round(knn_result[0], 2))
print("First 5 test predictions:", knn_pred[:5])



# SUPPORT VECTOR REGRESSION

svr_scaler = MinMaxScaler()

X_train_svr = svr_scaler.fit_transform(X_train)
X_test_svr = svr_scaler.transform(X_test)

# Changed to linear kernel
svr_model = SVR(kernel="linear", C=1000)
svr_model.fit(X_train_svr, y_train)

svr_pred = svr_model.predict(X_test_svr)

# Prediction using the same input
check_value_scaled = svr_scaler.transform(check_value)
svr_result = svr_model.predict(check_value_scaled)
```

```
print("\n========== SUPPORT VECTOR REGRESSION ==========")
print("Normalized Rural CPI input:", input_value)
print("Predicted Agricultural CPI:", round(svr_result[0], 2))
print("First 5 test predictions:", svr_pred[:5])

# SVR actual vs predicted plot
plt.figure(figsize=(8, 5))

plt.scatter(
    X_test['index_rl'],
    y_test,
    label='Actual CPI'
)

plt.scatter(
    X_test['index_rl'],
    svr_pred,
    label='Predicted CPI'
)

plt.title("SVR: Actual vs Predicted CPI")
plt.xlabel("Rural CPI")
plt.ylabel("Agricultural CPI")
plt.legend()
plt.show()


# FINAL COMPARISON OF ALL FOUR MODELS

comparison = pd.DataFrame({
    'Model': [
        'Linear Regression',
        'Decision Tree Regression',
        'KNN Regression',
        'Support Vector Regression'
    ],
    'MAE': [
        mean_absolute_error(y_test, lr_pred),
        mean_absolute_error(y_test, dt_pred),
        mean_absolute_error(y_test, knn_pred),
        mean_absolute_error(y_test, svr_pred)
    ],
    'MSE': [
        mean_squared_error(y_test, lr_pred),
        mean_squared_error(y_test, dt_pred),
        mean_squared_error(y_test, knn_pred),
        mean_squared_error(y_test, svr_pred)
    ],
    'R2 Score': [
        r2_score(y_test, lr_pred),
        r2_score(y_test, dt_pred),
        r2_score(y_test, knn_pred),
        r2_score(y_test, svr_pred)
    ]
})

print("\n========== FINAL MODEL COMPARISON ==========")
print(comparison.round(4).to_string(index=False))

print("\nBest model according to R2 Score:")
print(comparison.loc[comparison['R2 Score'].idxmax(), 'Model'])
```

7. References

[1] Ministry of Statistics and Programme Implementation (MOSPI), Government of India, "Consumer Price Index data," file cpialrl_5.csv. https://www.mospi.gov.in/ accessed

[2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[3] W. McKinney, "Data Structures for Statistical Computing in Python," in Proceedings of the 9th Python in Science Conference, 2010, pp. 56–61.

[4] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357–362, 2020.

[5] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.

[6] M. L. Waskom, "seaborn: statistical data visualization," Journal of Open Source Software, vol. 6, no. 60, p. 3021, 2021.

[7] T. Hastie, R. Tibshirani and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd ed. New York, NY, USA: Springer, 2009.

[8] L. Breiman, J. H. Friedman, R. A. Olshen and C. J. Stone, Classification and Regression Trees. Belmont, CA, USA: Wadsworth, 1984.

[9] T. Cover and P. Hart, "Nearest neighbor pattern classification," IEEE Transactions on Information Theory, vol. 13, no. 1, pp. 21–27, 1967.

[10] H. Drucker, C. J. C. Burges, L. Kaufman, A. Smola and V. Vapnik, "Support vector regression machines," in Advances in Neural Information Processing Systems 9, 1997.

[11] Scikit-learn developers, "scikit-learn User Guide and API Reference," https://scikit-learn.org/stable/, accessed [insert date].

[12] pandas development team, "pandas documentation," https://pandas.pydata.org/docs/, accessed [insert date].